Late Delivery Risk Prediction for Order Flow

Applied Science Internship Assignment 2026

Name: Negin Hadad

Email: negin.hadad@student.oulu.fi

LinkedIn:

www.linkedin.com/in/negin-hadad-8593221b7/

GitHub: github.com/Negin-Hadad

Problem & Decision Context

The problem I'm addressing:
Late deliveries create frustration for customers and inefficiencies for Wolt's operations. My goal is to predict, at the moment an order is placed, whether it is likely to be delivered late. This allows the business to proactively manage risk, for example by adjusting courier allocation or updating customer expectations.

Who uses this model?
The output would primarily support:

- Operations teams, to better allocate couriers during peak demand or low supply periods.
- Product managers, to understand patterns in delivery performance.
- Potentially automated systems that flag high-risk orders for intervention.

Decision-making supported
The model's probability score informs actionable decisions:

- Increase courier resources in high-risk areas or time windows
- Notify customers proactively about possible delays
- Adjust delivery ETA estimates dynamically

By framing the problem this way, I ensure that model outputs are directly connected to decisions, rather than being abstract predictions.

Why this problem matters?
Through exploratory analysis, I observed that delays are not random — they are influenced by order timing, courier availability, and environmental factors. Predicting these delays early can improve customer experience and operational efficiency, and help the business focus resources where they are most needed.

Data & EDA Findings

The data I used:
I worked with the simulated order flow dataset for Helsinki, which contains three months of order-level records. Each order includes:

- **Timestamp** of order placement
- **Order size** (number of items) and **category**
- **Estimated and actual delivery times**
- **Courier supply index** at the moment of order
- **Weather conditions** (precipitation)
- **Venue and customer locations** represented as H3 indices

This dataset captures the information available before the delivery starts, which is essential for making early predictions.

What surprised me! / implications for modeling

- Delay risk is **structured, not random**, which means the model can learn meaningful patterns.
- Signals that seem weak on their own (like precipitation or small ETA differences) become useful when combined with other features.
- Most of the predictive signals are **available at the time of order placement**, so early intervention is possible.

Key insights from exploratory analysis

- **Delivery delays vary over time**
  I found that late delivery rates fluctuate by hour of day and day of week, reflecting peak demand periods and operational load.

- **Courier supply strongly affects delays**
  Orders placed when courier availability is low are more likely to be late, making supply a key risk factor.

- **ETA uncertainty is informative**
  Orders with a larger difference between upper and lower ETA estimates tend to be late more often, suggesting that internal uncertainty already reflects operational risk.

- **Weather contributes modestly**
  Higher precipitation generally increases delay probability, especially during busy hours.

Feature Engineering & Representations

Turning raw data into meaningful signals

To make the model understand late delivery risk, I created features that capture **time patterns, order characteristics, and operational context**.

1 Time features

- I extracted the **hour of day** and **day of week** from the order timestamp.
- I created a **weekend indicator** to separate weekend orders, which tend to behave differently.
- I applied **cyclical encoding** to the hour (using sine and cosine transforms) to capture the continuous, repeating nature of time — for example, 23:00 is close to 0:00.

2 Order and ETA features

- **Item count** and **order category** reflect order complexity and handling requirements.
- I calculated the **ETA range** (upper - lower estimate) to represent the **uncertainty** already visible in Wolt's system.

3 Operational features

- **Courier supply index** measures real-time resource availability.
- **Precipitation** captures environmental risk.
- I also added a **same-hex flag** to indicate if the courier starts at the same location as the customer, which sometimes reduces delays.

4 Categorical handling

Categorical variables like **order category** and **H3 location indices** were encoded numerically so the model could process them.

!! Features not used

I avoided over-engineering features that could leak future information, like actual delivery times, to ensure the model only sees **pre-dispatch signals**.

Design rationale

By combining **temporal, operational, and uncertainty signals**, I designed features that the model can interpret to predict risk **at the moment an order is placed**.

Modeling Approach & Assumptions

- I approached this as a **risk prediction problem**, not a delivery time prediction task.
- The goal was not to estimate how long delivery would take, but to estimate **the probability that an order will be late**, given only the information available at order time.

Modeling approach

I built a **baseline probabilistic model** using logistic regression, chosen intentionally for:

- **Interpretability** → clear understanding of what drives risk
- **Stability** → reliable behavior on structured operational data
- **Transparency** → easy to explain to non-technical stakeholders
- **Calibration** → meaningful probability outputs, not just class labels

Rather than optimizing for raw accuracy, I focused on **risk ranking** and **decision usefulness**, meaning the model should tell ***which orders are risky***, not just whether they are delayed.

Framing decision

- This model is designed as a **risk scoring system**, not a final predictor:
- A tool for prioritization, monitoring, and intervention, not just classification.
- It answers the question:
  "Which orders should I worry about the most right now?"

Core assumptions

Risk is probabilistic, not deterministic
Delays are influenced by many stochastic factors (traffic, demand surges, courier availability).
Therefore, prediction should express **uncertainty**, not certainty.

Operational signals carry predictive power
Supply–demand imbalance, ETA uncertainty, time patterns, and spatial proximity are meaningful early indicators of delay risk.

Interpretability matters more than complexity (at baseline stage)
A transparent model is more valuable early on than a complex black-box model.

Decisions happen at order time
All features are restricted to information available at order placement — no future leakage.

Model Training & Evaluation Strategy

I treated this as a **real operational prediction problem**, not a random offline ML task. That directly influenced how I split the data, trained the model, and evaluated performance.

Data splitting strategy (time-aware)

Instead of random splitting, I used a **temporal split**:

- **70% Training** → past data
- **15% Validation** → future data (model tuning & thresholding)
- **15% Test** → unseen future data (final evaluation)

This simulates real deployment. So, the model is always trained on the past and predicts the future.
It avoids **temporal leakage** and unrealistic performance inflation.

Training setup

I trained a **logistic regression model** with:

- Feature scaling (standardization)
- Class balancing (class_weight="balanced") to handle delay imbalance
- Probabilistic output (not hard classification)

The model outputs a **delay probability**, not just a label.

Evaluation philosophy

I did not evaluate the model as a simple classifier. I evaluated it as a **risk ranking system**.

Metrics used:

- **ROC-AUC** → How well the model ranks risky orders vs safe ones
- **PR-AUC** → Performance under class imbalance (delays are minority class)
- These metrics answer: "Does the model meaningfully separate risky orders from safe ones?"

Decision threshold strategy

Instead of using a default 0.5 threshold, I used a **recall-driven threshold**:

- **Target: ~70% recall on delayed orders**
- This means the model is tuned to **catch most risky orders**, even if it creates some false alarms.
- This reflects a real operational logic: *Missing a late delivery is more costly than flagging a safe one.*
- So the model is optimized for **risk sensitivity**, not comfort.

Framing

- The evaluation is not: "Is the model perfect?"
- It is: "Is the model useful for early risk detection and intervention?"
- This is the difference between **ML benchmarking** and **operational ML**.

Limitations & Production Risks

Key Constraints & Risks

This model is intentionally simple and interpretable, but that comes with clear limitations.

LINEAR MODEL LIMITATIONS

Because I rely on a **linear model (logistic regression)**, the system can only capture **simple, additive relationships**. Real delivery dynamics are often non-linear (e.g. interactions between time, weather, supply, and demand), so some patterns are inevitably missed.

GENERALIZATION RISKS

The model is also trained on a **single seasonal dataset**, which means it may not generalize well to:

- different cities
- different seasons
- changing customer behavior
- evolving courier supply patterns

DISTRIBUTION SHIFT

In production, this would likely break under **distribution shift**:

- sudden demand spikes
- extreme weather events
- operational policy changes
- platform growth or geographic expansion

DATA AVAILABILITY

Another limitation is **data availability**:
If real-time features like courier supply or weather are delayed, noisy, or missing, prediction quality will degrade quickly.

In short: This system is reliable as a **risk-flagging tool**, but not as a fully autonomous decision engine. It works best as a **support layer** for operations, not as a standalone control system.

Next Steps & System Evolution

If this system were deployed in production, my focus would shift from building a model to building a learning system. The first step would be moving beyond a linear baseline to **non-linear models** (e.g. tree-based models or neural networks) to capture complex interactions between time, demand, supply, weather, and geography.

I would then introduce **continuous learning**:

- periodic retraining
- data drift monitoring
- performance decay detection
- automatic recalibration of thresholds

From a system perspective, I would integrate **real-time signals**:

- live courier availability
- traffic conditions
- dynamic weather feeds
- real-time demand intensity

Operationally, I would connect predictions to **decision workflows**:

- dynamic ETA adjustment
- proactive courier allocation
- risk-based order prioritization
- customer communication strategies

Finally, I would evaluate impact not only with ML metrics, but with **business metrics**:

- customer satisfaction
- cancellation rates
- courier efficiency
- SLA compliance

The long-term goal:
Not a better model — but a smarter delivery system that learns, adapts, and improves continuously.